Ideally, we index our documents with of `age` as an integer value. However, a user may accidentally set the value of `age` as a string type (`"2"`) and invoke the call.

Listing 4.8 Setting a string value for a numeric field

```
PUT cars/_doc/1
{
  "make":"BMW",
  "model":"X3",
  "age":"2"
}
```

An integer input as a string

Elasticsearch indexes this document without any errors. Although the `age` field is expected to contain an integer, Elasticsearch *coerces* the type for us to avoid errors. The coercion only works if the parsed value of the field matches the expected data type. In this case, parsing `"2"` results in `2`, which is a number.

So far, we've talked a lot about mapping and data types without putting much emphasis on data types. Designing your schema with appropriate data types is crucial in optimizing the search experience. We need to understand data types, their characteristics, and when to use them. Unlike a database or programming language, Elasticsearch comes with a long list of data types, catering to almost every data shape and form. The next section looks at data types in detail.

4.4 Data types

Similar to a variable's data type in a programming language, fields in a document have a specific data type associated with them. Elasticsearch provides a rich list of data types, from simple to complex to specialized. The list of these types keeps growing, so watch for more.

Elasticsearch provides over two dozen different data types, so we have a good selection of appropriate data types based on our specific needs. Data types can be broadly classified in the following categories:

- *Simple types*—Common data types representing strings (textual information), dates, numbers, and other basic data variants. Examples are `text`, `boolean`, `long`, `date`, `double`, and `binary`.
- *Complex types*—Created by composing additional types, similar to an object construction in a programming language where objects can hold inner objects. Complex types can be flattened or nested to create even more complex data structures based on the requirements. Examples are `object`, `nested`, `flattened`, and `join`.
- *Specialized types*—Used predominantly for specialized cases such as geolocation and IP addresses. Examples are `geo_shape`, `geo_point`, and `ip`, and range types such as `date_range` and `ip_range`.

NOTE The full range of types is available from the official documentation: http://mng.bz/yQpe.